COP290 Assignment 3 — LevelDB Modifications

Range Scan, Range Delete, and Manual Full Compaction

Angadi Charan Umesh (<2024CS10954>)
Kunal Kumar (<2024CS10034>)

Course: COP290 – Design Practices
Topic: Modifying a large code base (LevelDB)
Repository: https://github.com/cs1240954/COP290_Assignment3.git

Contents

1 Introduction **2**

2 LevelDB Internals: Write Path and Compaction **3**

2.1 Storage layout and the LSM hierarchy . . . 3

2.2 Write path . . . 3

2.3 Read path (relevant for `Scan`) . . . 4

2.4 Compaction mechanism . . . 5

3 Range Scan: Design and Implementation **5**

3.1 Goal and required semantics . . . 5

3.2 Why iterator reuse is the right approach . . . 6

3.3 Implementation . . . 6

3.4 Correctness analysis . . . 6

4 Range Delete: Design and Implementation **7**

4.1 Goal and required semantics . . . 7

4.2 Design decision . . . 7

4.3 Implementation . . . 8

4.4 Correctness analysis . . . 9

5 Manual Full Compaction: Design and Implementation **9**

5.1 Goal and required semantics . . . 9
5.2 State variables added to `DBImpl` . . . 10
5.3 The gate: `WaitIfFullCompaction` . . . 10
5.4 The compaction sweep . . . 11
5.5 Statistics collection . . . 12
5.6 Correctness analysis . . . 13

6 Evaluation **13**
6.1 Tests we wrote . . . 13
6.2 Compaction statistics: format . . . 14
6.3 Compaction statistics: interpretation . . . 14
6.4 Behaviour of `Scan` and `DeleteRange` . . . 14

7 Challenges and Lessons Learned **15**

8 Conclusion **15**

References **15**

1 Introduction

LevelDB is an embedded, ordered key–value store written in C++ at Google. It does not run as a server; instead, an application links against it as a library and uses the `leveldb::DB` interface to read and write opaque byte strings. Because keys are kept ordered by a comparator (lexicographic by default), the library can support both point lookups and range traversal efficiently.

Internally, LevelDB is a Log-Structured Merge (LSM) tree. New writes are buffered in an in-memory structure (the *MemTable*), persisted as immutable on-disk files (*SSTables*), and reorganised over time by a process called *compaction*. The main strength of this design is that random user writes turn into a few large sequential disk writes; the cost is that reads may have to consult several files, and the engine continually has to merge and prune data in the background.

For this assignment we extended the LevelDB codebase with three new public APIs:

- `DB::Scan` – return all key–value pairs whose keys fall in the half-open interval $[start_key, end_key)$, with the same visibility rules as `Get`.
- `DB::DeleteRange` – logically delete every key in $[start_key, end_key)$, with physical removal happening during compaction.
- `DB::ForceFullCompaction` – run a synchronous, level-by-level compaction sweep, blocking

foreground operations during the call, and print five compaction statistics in a human-readable form.

A large part of the work was *understanding* LevelDB before changing it. The codebase is roughly thirty thousand lines of C++ across `db/`, `table/`, `util/` and `include/leveldb/`; rather than reading it end-to-end, we traced the control flow of a single `Put`, a single `Get`, and one background compaction — enough mental model to plug the new APIs into the existing machinery without inventing a parallel system. The rest of the report follows the assignment's deliverables: the write/read/compaction analysis (§2); the design and implementation of `Scan` (§3), `DeleteRange` (§4) and `ForceFullCompaction` (§5); and the evaluation including compaction statistics (§6).

2 LevelDB Internals: Write Path and Compaction

This section is grounded in the actual source files we read while writing the new APIs. We focus on the parts that the assignment explicitly asks about: the write path, the read path, and the compaction mechanism. The same files are reused by `Scan`, `DeleteRange`, and `ForceFullCompaction`, so the mental model carries directly into our implementation sections later.

2.1 Storage layout and the LSM hierarchy

The on-disk state is a collection of immutable SSTable files organised across levels. From `db/dbformat.h` and `db/version_set.cc`: `config::kNumLevels` is 7, `config::kL0_CompactionTrigger` is 4, and level byte budgets grow roughly tenfold. Level 0 files may have overlapping key ranges (each file is a separate MemTable flush), while levels 1 through 6 keep the invariant that no two files in the same level overlap; this bounds a point lookup to at most one file per level beyond level 0.

The live file set per level is tracked by a `Version` object inside `VersionSet` (`db/version_set.h`). Every change is recorded as a `VersionEdit` appended to the `MANIFEST` via `VersionSet::LogAndApply`. Active iterators hold reference counts on a `Version`, which is why compaction can install a new `Version` without yanking the floor out from under in-flight readers.

2.2 Write path

Files used: `db/db_impl.cc`, `db/write_batch.cc`, `db/log_writer.cc`, `db/memtable.cc`, `db/skiplist.h`.

1. **API entry.** A user call to `DB::Put(key, value)` or `DB::Delete(key)` is implemented in the base class as a one-entry `WriteBatch`, which is then forwarded to `DBImpl::Write`. `WriteBatch` is just a contiguous byte buffer encoding $\langle type, key, value \rangle$ records, where `type` is either `kTypeValue` or `kTypeDeletion` (see `db/write_batch.cc` and `db/dbformat.h`). Because `Put` and `Delete` share the same path, deletions cost the same as writes and are

recoverable from the WAL.

2. **Writer queue and group commit.** `DBImpl::Write` pushes a `Writer` struct onto an internal `std::deque` and waits on a condition variable until the thread reaches the front of the queue. The thread that becomes the leader calls `BuildBatchGroup`, which merges as many waiting batches as it safely can into a single batch. This is LevelDB's group-commit optimisation: a hundred concurrent `Put` calls do not produce a hundred WAL appends; they produce one. The detail that surprised us was that the leader does the I/O on behalf of the followers, then wakes them with the same status.

3. **Backpressure (`MakeRoomForWrite`).** Before a write is applied, the leader calls `MakeRoomForWrite`. This routine decides whether the active MemTable still has room, whether the previous immutable MemTable has finished flushing, and whether level 0 has too many files. The relevant constants are `kL0_SlowdownWritesTrigger` (writes are throttled by a 1 ms sleep) and `kL0_StopWritesTrigger` (writes block until compaction catches up). When the active MemTable is full, this function rotates `mem_` into `imm_`, opens a new log file and MemTable, and signals the background thread.

4. **WAL append and MemTable insertion.** With space ensured, the leader releases `mutex_`, appends the (possibly merged) batch to the WAL through `log::Writer::AddRecord`, optionally `Sync()`s the file if `WriteOptions::sync` is set, and then applies the batch to the MemTable via `WriteBatchInternal::InsertInto`. Each entry inserted into the MemTable carries a fresh `SequenceNumber`; this number is the single most important value in LevelDB's concurrency story, because it is what snapshots, iterators and compaction all key off of.

5. **Minor compaction (flush of `imm_`).** When the background thread sees a non-null `imm_`, it calls `DBImpl::CompactMemTable`, which writes a level-0 SSTable through `WriteLevel0Table` and `BuildTable` (`db/builder.cc`). The new file is registered with a `VersionEdit` via `LogAndApply`. After this point the immutable MemTable is unreferenced and freed.

The high-level lecture statement that "writes go to the MemTable first" matches the implementation almost exactly — but the WAL plus group-commit plus backpressure handling are the parts that are not obvious from the diagrams and that we had to read carefully.

2.3 Read path (relevant for `Scan`)

Files used: `db/db_impl.cc`, `db/db_iter.cc`, `db/version_set.cc`, `db/table_cache.cc`.

`DBImpl::Get` captures the current sequence number under `mutex_`, pins references to `mem_`, `imm_` and the current `Version*`, and then does the actual lookup unlocked in that same order. `Version::Get` reads level 0 newest-first (because files there may overlap) and binary-searches deeper levels; SSTable blocks come through `TableCache` / `BlockCache`.

For range traversal, `DBImpl::NewIterator` builds a `MergingIterator` over `mem_`, `imm_` and the per-level SSTable iterators, and wraps it in `NewDBIterator` (`db/db_iter.cc`). The `DBIter` layer walks internal keys in order, collapses multiple versions of the same user key to the latest visible one using the snapshot sequence number, and skips `kTypeDeletion` entries. We rely on this behaviour directly in `Scan` and did not have to re-implement merging or visibility.

2.4 Compaction mechanism

Files used: `db/db_impl.cc` (`BackgroundCompaction`, `DoCompactionWork`), `db/version_set.cc` (`PickCompaction`, `CompactRange`, `NeedsCompaction`).

SSTables are immutable, so an update never rewrites an old file in place; it adds a newer record at a higher sequence number, and the old one is reclaimed only when files are merged. `DBImpl::MaybeScheduleCompaction` schedules background work, which runs on a single background thread. The relevant function `DBImpl::BackgroundCompaction` has three branches we had to track:

1. **Memtable flush.** If `imm_` is non-null, it is flushed first via `CompactMemTable`.
2. **Trivial move.** If the chosen compaction has exactly one input file in level L, no overlap with level $L{+}1$, and no overlap with the grandparent level, the file is moved without re-writing. The per-level file list is updated and that is all.
3. **Real merge (`DoCompactionWork`).** Otherwise, a merging iterator is built over the input files and the output is written as new SSTables in level $L{+}1$. `DoCompactionWork` keeps only the most recent version of each user key visible to all active snapshots, drops `kTypeDeletion` tombstones once it is safe to do so (no live snapshot needs them and no deeper level still contains the key), and stops the current output file when its size or its "grandparent overlap" grows too large.

This three-way split is the reason our `ForceFullCompaction` (§5) accumulates statistics from *three* places: flushes, trivial moves, and real merges. Input files are picked by `VersionSet::PickCompaction` (size- and seek-driven) or `VersionSet::CompactRange` for manual compactions; either way the result is a `Compaction` object spanning levels L and $L{+}1$.

3 Range Scan: Design and Implementation

3.1 Goal and required semantics

The assignment requires the API

```
Status DB::Scan(const ReadOptions&,
               const Slice& start_key,
               const Slice& end_key,
               std::vector<std::pair<std::string,std::string>>* result);
```

returning all entries in the half-open interval $[start_key, end_key)$, in ascending key order, with the same visibility rules as `Get` (including snapshots from `ReadOptions::snapshot`). An inverted or empty range must produce an empty result, not an error.

3.2 Why iterator reuse is the right approach

The naive alternative would be to call `Get` for every candidate user key. That is wrong on two counts: it cannot enumerate keys you do not already know, and it costs $O(K \log N)$ for K results in a database of N keys, which throws away the ordered structure LevelDB already maintains.

LevelDB already exposes an iterator that merges memtable and SSTable sources and implements snapshot-aware visibility (`db/db_iter.cc`, `NewDBIterator`). Reusing that iterator gives us $O(\log N + K)$ behaviour and free correctness for visibility, deletion handling, and snapshot semantics. The only thing left for `Scan` to do is bound the walk on the user-key axis.

3.3 Implementation

`DBImpl::Scan` is implemented in `db/db_impl.cc`. The structure is straightforward and intentionally so:

```
Status DBImpl::Scan(const ReadOptions& options,
                    const Slice& start_key, const Slice& end_key,
                    std::vector<std::pair<std::string,std::string>>* result) {
  WaitIfFullCompaction();
  if (result == nullptr) {
    return Status::InvalidArgument("result is null");
  }
  result->clear();
  const Comparator* cmp = user_comparator();
  if (cmp->Compare(start_key, end_key) >= 0) {
    return Status::OK(); // empty / inverted range
  }
  Iterator* it = NewIterator(options); // honours options.snapshot
  for (it->Seek(start_key); it->Valid(); it->Next()) {
    Slice k = it->key();
    if (cmp->Compare(k, end_key) >= 0) break; // half-open upper bound
    result->push_back(std::make_pair(k.ToString(), it->value().ToString()));
  }
  Status s = it->status();
  delete it;
  return s;
}
```

The matching public declaration is added in `include/leveldb/db.h`, the override declaration in `db/db_impl.h`, and a stub of the same shape in the test mock `ModelDB` inside `db/db_test.cc`.

3.4 Correctness analysis

Snapshot semantics. `NewIterator` forwards `options.snapshot` into `NewDBIterator`, which only surfaces internal keys whose sequence number is at most the snapshot bound. So if the caller passes a snapshot, the scan reflects the DB at that snapshot regardless of concurrent

writes; if not, the iterator captures the current last sequence number at construction (same as `Get` without a snapshot).

Half-open interval and tombstones. The break condition uses the user comparator, so equality with `end_key` is treated as out-of-range and custom orderings are respected. Because `DBIter` already filters `kTypeDeletion` entries and older versions, `result` contains exactly one pair per visible user key without any extra work on our side.

Edge cases and resources. If `start_key` $\geq$ `end_key`, we return `Status::OK()` with an empty vector before constructing an iterator. Otherwise we always check `it->status()` and propagate any read-path error before `delete it;`, which releases the references on the MemTable and `Version`.

Concurrency. `WaitIfFullCompaction` (§5) blocks `Scan` while a manual full compaction is running, removing the only case where an iterator would race the file rewrites of a full sweep.

4 Range Delete: Design and Implementation

4.1 Goal and required semantics

The required API is

```
Status DB::DeleteRange(const WriteOptions&,
                       const Slice& start_key,
                       const Slice& end_key);
```

and the assignment text says: "Keys that fall within an active deletion range must be discarded when SSTables are merged during compaction and must not appear in the resulting SSTables." That is, the operation is logical at write time and physical at compaction time, exactly like single-key `Delete`.

4.2 Design decision

We considered two broad designs.

Design A: persistent range tombstone. A `DeleteRange` call writes one *interval* marker. Future writes that fall inside the interval are suppressed by the read/compaction logic until a new write at a higher sequence number is supposed to override the interval, the interval expires, etc. This is the design used in some other LSM-based engines, and it is powerful: a single record can shadow an arbitrarily large key range, including keys that do not yet exist.

Design B: per-key tombstones built from the live key set. A `DeleteRange` call enumerates the keys currently visible in the range and writes one ordinary `kTypeDeletion` tombstone per key, all in a single `WriteBatch`. The compaction code then strips them later, just like for ordinary `Delete`.

We chose **Design B**. LevelDB's on-disk format and `WriteBatch` encoding only know two record types — `kTypeValue` and `kTypeDeletion` (`db/dbformat.h`) — so adding a new "range tombstone" type would cascade into `WriteBatch`, `MemTable`, the SSTable format, `DBIter` merging, and the snapshot rules in `DoCompactionWork`. Since the TA clarified on Piazza that `DeleteRange` is graded only on correctness (not efficiency), the simpler design that reuses `NewIterator`, `WriteBatch`, `Write` and `DoCompactionWork` was strictly safer.

The user-visible consequence is that our `DeleteRange` is *not* a permanent ban on a key interval: a later `Put` for a key in the range gets a higher sequence number and becomes visible again, exactly as in `Delete(k); Put(k, v)`. This is a deliberate choice and we test it explicitly (§6).

4.3 Implementation

`DBImpl::DeleteRange` lives in `db/db_impl.cc`. The structure is:

```
Status DBImpl::DeleteRange(const WriteOptions& options,
                           const Slice& start_key, const Slice& end_key) {
  WaitIfFullCompaction();
  const Comparator* cmp = user_comparator();
  if (cmp->Compare(start_key, end_key) >= 0) {
    return Status::OK(); // empty or inverted -> nothing to do
  }

  const int kMaxPasses = 512;
  int pass = 0;
  while (pass < kMaxPasses) {
    std::vector<std::string> keys;
    Iterator* it = NewIterator(ReadOptions());
    it->Seek(start_key);
    bool stop = false;
    while (it->Valid()) {
      Slice k = it->key();
      if (cmp->Compare(k, end_key) >= 0) {
        stop = true;
      } else {
        keys.push_back(k.ToString());
        it->Next();
      }
      if (stop) break;
    }
    Status scan_status = it->status();
    delete it;
    if (!scan_status.ok()) return scan_status;
    if (keys.empty()) return Status::OK();

    WriteBatch batch;
    for (size_t i = 0; i < keys.size(); ++i) {
      batch.Delete(Slice(keys[i]));
    }
```

```
    Status s = Write(options, &batch);
    if (!s.ok()) return s;
    pass++;
  }
  return Status::Corruption("DeleteRange: range did not empty after retries");
}
```

The bounded re-scan loop deserves a comment. In a single-threaded test, one pass is always enough: we enumerate the keys, write a batch of tombstones, and we are done. Under concurrent writers, however, another thread can insert a new key into the range between our `Seek` and our `Write`. The retry loop gives the operation a way to converge — it is bounded because we did not want a malicious or buggy concurrent writer to spin `DeleteRange` forever, and the cap of 512 passes is high enough that it has never been hit in any of our tests. It is conservative rather than clever, and that suited the "correctness over efficiency" framing of the assignment.

4.4 Correctness analysis

Snapshot-consistent discovery and per-batch atomicity. The iterator at the top of each pass captures a sequence number under the mutex and surfaces keys under standard `DBIter` rules; any key inserted by a parallel writer afterwards is invisible to this pass but caught by the next. The subsequent `Write` appends the batch to the WAL as one record and then applies it to the MemTable as a contiguous sequence-number range (`WriteBatchInternal::InsertInto`), so a concurrent snapshot reader sees either all tombstones from the pass or none.

Iterator lifetime and compaction interaction. We delete the iterator before `Write`; otherwise we would hold references into the version chain while `Write` potentially rotates the memtable and schedules a compaction — a quiet form of use-after-free. The tombstones we emit are identical to those of `DB::Delete`, so `DoCompactionWork` drops them under the usual rule (older than every active snapshot and no deeper level still contains the user key). A later `Put` of the same key gets a higher sequence number and becomes visible again.

Concurrency with full compaction. The `WaitIfFullCompaction` call at the top serialises `DeleteRange` with manual full compactions, so we never enumerate keys while `ForceFullCompaction` is rewriting files.

5 Manual Full Compaction: Design and Implementation

5.1 Goal and required semantics

The required API is

```
Status DB::ForceFullCompaction();
```

The assignment text and the Piazza clarifications imply, together:

- the call is synchronous — it returns only after the full compaction has finished;

- foreground reads/writes *may* be blocked during the call;
- compaction is triggered level by level starting from level 0 and moving downward;
- there must be no "meaningless pushdown" — if a level has no useful compaction work, no SSTable should be pushed down just for the sake of it;
- five compaction statistics (compactions executed, input files, output files, bytes read, bytes written) must be printed in human-readable form on completion.

5.2 State variables added to `DBImpl`

In `db/db_impl.h` we added:

```
port::Mutex full_compact_mu_;
port::CondVar full_compact_cv_;
bool full_compaction_active_;

struct FullCompactionStats {
  int compaction_count = 0;
  int64_t input_files = 0;
  int64_t output_files = 0;
  int64_t bytes_read = 0;
  int64_t bytes_written = 0;
};
FullCompactionStats full_compaction_stats_;
FullCompactionStats* full_compaction_stats_collector_; // non-null while collecting
```

We use a separate `full_compact_mu_` / `full_compact_cv_` pair as the gate rather than the main `mutex_`, because `mutex_` is held only for short critical sections inside `DBImpl` and a long "full compaction in progress" wait on it would be intrusive. A `thread_local bool tls_in_force_full_compaction` (file-scope in `db/db_impl.cc`) prevents self-deadlock: the running thread re-enters public APIs such as `Write` during `TEST_CompactMemTable`, all of which call `WaitIfFullCompaction()`, and would otherwise block on its own gate.

5.3 The gate: `WaitIfFullCompaction`

```
void DBImpl::WaitIfFullCompaction() {
  if (tls_in_force_full_compaction) return; // skip on the running thread
  MutexLock l(&full_compact_mu_);
  while (full_compaction_active_) {
    full_compact_cv_.Wait();
  }
}
```

This call is added at the top of every public foreground entry point that can observe or modify the database while files are being rewritten: `Get`, `Write`, `NewIterator`, `Scan`, `DeleteRange`, `CompactRange`, `GetSnapshot`, `ReleaseSnapshot`, `GetProperty`, `GetApproximateSizes`,

`RecordReadSample`, and at the start of `ForceFullCompaction` itself (so two threads cannot run two manual compactions on top of each other).

5.4 The compaction sweep

`ForceFullCompaction` (`db/db_impl.cc`) does the following:

```
Status DBImpl::ForceFullCompaction() {
  WaitIfFullCompaction(); // block on any prior manual full compaction
  tls_in_force_full_compaction = true; // skip our own gate from now on

  full_compact_mu_.Lock();
  full_compaction_active_ = true;
  full_compact_mu_.Unlock();

  Status status = Status::OK();
  {
    MutexLock l(&mutex_);
    full_compaction_stats_ = FullCompactionStats();
    full_compaction_stats_collector_ = &full_compaction_stats_;
  }

  status = TEST_CompactMemTable(); // (1) flush memtable -> level 0

  // (2) walk levels 0..kNumLevels-2; stop at a level when no progress
  for (int level = 0;
       status.ok() && level + 1 < config::kNumLevels;
       ++level) {
    for (int attempts = 0; status.ok() && attempts < 100; ++attempts) {
      int before = 0;
      { MutexLock l(&mutex_); before = full_compaction_stats_.compaction_count; }
      TEST_CompactRange(level, nullptr, nullptr);
      MutexLock l(&mutex_);
      if (!bg_error_.ok()) { status = bg_error_; break; }
      if (full_compaction_stats_.compaction_count == before) break; // nothing
          happened
    }
  }

  {
    MutexLock l(&mutex_);
    const FullCompactionStats st = full_compaction_stats_;
    full_compaction_stats_collector_ = nullptr;
    if (status.ok() && !bg_error_.ok()) status = bg_error_;
    std::fprintf(stdout,
        "Number of compactions executed: %d\n"
        "Number of input files: %lld\n"
        "Number of output files: %lld\n"
        "Total bytes read during compaction: %lld\n"
```

```
        "Total bytes written during compaction: %lld\n",
        st.compaction_count,
        (long long)st.input_files, (long long)st.output_files,
        (long long)st.bytes_read, (long long)st.bytes_written);
  }

  full_compact_mu_.Lock();
  full_compaction_active_ = false;
  full_compact_mu_.Unlock();
  full_compact_cv_.SignalAll();
  tls_in_force_full_compaction = false;
  return status;
}
```

A few choices in this code are worth pointing out. We reuse existing internals — `TEST_CompactMemTable` and `TEST_CompactRange` are already the synchronous entry points wired to `ManualCompaction` and `BackgroundCompaction`, so we did not need a parallel compaction engine. The inner loop avoids meaningless pushdown by re-invoking `TEST_CompactRange` only while `compaction_count` grows; when a level has no work, `VersionSet::CompactRange` returns `nullptr`, `BackgroundCompaction` does nothing, and our loop observes the absence of progress and moves on. The outer bound `level + 1 < kNumLevels` skips the deepest level (no $L{+}1$ to merge into), and the inner cap of 100 attempts per level is a safety belt — in practice each level finishes in 1–3 attempts.

5.5 Statistics collection

The five statistics are accumulated into `full_compaction_stats_` from three places in `db/db_impl.cc`:

1. **Memtable flush (`WriteLevel0Table`).** When `imm_` is flushed during a forced compaction, we contribute one compaction event, one output file, and the SSTable's byte size to both `bytes_read` and `bytes_written` (the flush consumes the memtable and produces a file).
2. **Trivial moves (`BackgroundCompaction`).** One input file becomes one output file at $L{+}1$ without rewriting; we charge one compaction, one input, one output, and the file size to both byte counters.
3. **Real merge (`DoCompactionWork`).** The merge path already keeps per-compaction byte counts (used by `leveldb.stats`); we piggy-back on that accounting and add `compaction_count`, `input_files`, `output_files`, `bytes_read` and `bytes_written` into `full_compaction_stats_` only when `full_compaction_stats_collector_` is non-null.

The collector pointer keeps statistics opt-in: background compactions triggered by automatic heuristics (L0 file count, level size, seek hints) never contribute. The five-line output is printed under `mutex_` after the level cascade and before the gate is released, so any thread waking up on `WaitIfFullCompaction` sees a database whose stats have already been emitted.

5.6 Correctness analysis

Serialisation and gating. Two threads calling `ForceFullCompaction` serialise on `full_compact_mu_` via `WaitIfFullCompaction()`: the second waits on `full_compact_cv_` until the first finishes and then runs its own sweep with reset statistics. While `full_compaction_active_` is true, every public read/write API parks at the same gate, so foreground threads cannot observe a torn state during file rewrites; only the running thread touches the DB, and it bypasses the gate via the thread-local flag.

No deadlock, errors not lost. The thread-local guard keeps our own internal calls to `Write` and `NewIterator` (used inside `TEST_CompactMemTable`) from blocking on the gate they opened. Inside the level loop, every re-acquisition of `mutex_` copies out `bg_error_` and breaks on the first failure, so partial work and the error reach the caller together.

Termination. The loop terminates because each inner iteration either records a strictly greater `compaction_count` (real progress) or exits on the no-progress check; once all levels are processed or a background error is observed, the function returns.

6 Evaluation

6.1 Tests we wrote

We added three focused tests in `db/db_test.cc` under the prefix `DBTest.COP290_*`, on top of the standard LevelDB test fixture so that they share its setup/teardown logic and use a real database instance:

- `COP290_ScanDeleteRangeForceCompaction`: writes a few keys, scans them, deletes a sub-range, scans again, and finally invokes `ForceFullCompaction()` so that the printed statistics line is exercised at least once.
- `COP290_ScanDeleteRangeEdgeCases`: covers `start_key == end_key`, an inverted range, and a range that covers every key in the database, asserting that the result vector and the post-state of the DB are correct in each case.
- `COP290_ScanWithSnapshot`: writes a key, takes a snapshot, overwrites the key, then performs two scans — one with the snapshot and one without — and checks that the snapshot scan returns the old value while the live scan returns the new value.

We also extended the in-memory `ModelDB` (the test harness's reference implementation) so that it correctly mirrors the new APIs at the model level. For `ModelDB::Scan`, we filter the model's `std::map` by the snapshot sequence number; for `ModelDB::DeleteRange`, we erase keys in the requested range; for `ModelDB::ForceFullCompaction`, we print a five-line stub so that the test process produces the same kind of output regardless of which DB the test is running against.

The full `leveldb_tests` binary still passes after our changes, which gave us confidence that we did not break any existing invariants while wiring the new code in.

6.2 Compaction statistics: format

When `ForceFullCompaction` returns, our implementation prints, on `stdout`:

```
Number of compactions executed: <K>
Number of input files: <I>
Number of output files: <O>
Total bytes read during compaction: <BR>
Total bytes written during compaction: <BW>
```

This matches the human-readable format required by the assignment statement. The labels are intentionally verbose — they are easy for a grader to grep for and unambiguous about which number is which.

6.3 Compaction statistics: interpretation

We ran the API on a few workload shapes (small in-memory only; medium with a handful of SSTables; medium with concurrent writers and `DeleteRange` calls). Three patterns showed up consistently. First, *compactions* ≥ 1 as soon as any data exists — the first contribution is almost always the memtable flush in `WriteLevel0Table`, so even tiny runs that never reach level 1 still report at least one "compaction" instead of an all-zero line that would look broken. Second, *bytes_read* $\approx$ *bytes_written* for trivial moves (nothing is rewritten), while *bytes_read* $>$ *bytes_written* for real merges — the gap is a rough measure of physical reclamation when the workload had overwrites or deletes. Third, different workload timings produce different byte totals even for the same logical operations, because the stats describe physical work, not logical operations. While debugging against a classmate's unofficial checker, our functional outputs matched, but byte totals occasionally differed because the two implementations batched `DeleteRange` differently and therefore produced different SSTable layouts — an implementation choice, not a correctness issue.

6.4 Behaviour of `Scan` and `DeleteRange`

`Scan` returns keys in ascending order, exactly the keys k with `start_key` $\leq k <$ `end_key` that are visible at the read snapshot, and matches `Get` key-by-key; empty and inverted ranges return an empty vector. `DeleteRange` causes subsequent `Get` / `Scan` calls to skip the affected keys immediately, before any compaction; after a subsequent `ForceFullCompaction`, the SSTables on disk no longer contain the deleted entries (we verified this by reading `leveldb.sstables` via `GetProperty` before and after). Issuing a `Put` for a key in a previously deleted range and reading it back returns the new value, confirming that the per-key tombstone design does not block future writes.

7 Challenges and Lessons Learned

Navigating the codebase and compaction's many shapes. The hardest part of the first week was just not getting lost. `DBImpl::Write` alone routes through six other methods, each of which can recursively schedule background work, and `BackgroundCompaction` turned out to be three different routines stitched together (memtable flush, trivial move, real merge). Reading `db_impl.cc`, `version_set.cc` and `db_iter.cc` side by side made the control flow click; once we saw the three-way split, the stats hooks for `ForceFullCompaction` fell into place because they had to live in all three places.

Iterators, sequence numbers, and concurrency. Scan-style operations raised subtle questions — which sequence number the iterator sees, what happens to writes during iteration, what "latest visible version" means when a deletion is involved. Reading `DBIter::FindNextUserEntry` was when those edge cases collapsed into one rule. The Piazza clarification meant we were not asked to add concurrency to LevelDB, only not to break it; every public API we added uses `WaitIfFullCompaction` before touching state, every iterator is deleted before its corresponding write, and we never extended the lifetime of a `Version` or MemTable beyond what the original code does.

Debugging the statistics. Functional tests passed early, but matching the exact byte numbers from an unofficial classmate-built checker took several rounds. Each round taught us where in the codebase a particular byte is actually counted; by the end we had a clear picture of what each number means and why two correct implementations can still produce slightly different byte totals — and we documented that explicitly rather than hiding it.

8 Conclusion

The three new APIs hook into LevelDB's existing design rather than fighting it: `Scan` reuses the merging iterator and snapshot machinery, `DeleteRange` reuses `WriteBatch`, `Write` and the tombstone path through `DoCompactionWork`, and `ForceFullCompaction` reuses `TEST_CompactMemTable`, `TEST_CompactRange`, `ManualCompaction` and the per-compaction byte counters already in the codebase, adding only the gating, level cascade and statistics aggregation the assignment requires. The work also taught us how to read a non-trivial systems codebase — after time inside `db_impl.cc`, `version_set.cc` and `db_iter.cc`, the LSM-tree picture from class became something we could connect to specific functions, mutexes and sequence-number rules instead of a diagram.

References (LevelDB source files used)

- `include/leveldb/db.h` – public DB interface; added `Scan`, `DeleteRange`, `ForceFullCompaction` declarations.
- `db/db_impl.h`, `db/db_impl.cc` – main DB logic; added `Scan`, `DeleteRange`, `ForceFullCompaction`, `WaitIfFullCompaction`, full-compaction stats, and statistics hooks

in `WriteLevel0Table`, `BackgroundCompaction`, `DoCompactionWork`.

- `db/db_iter.cc` – `DBIter` merge and visibility logic used (unchanged) by our `Scan`.
- `db/version_set.h`, `db/version_set.cc` – `VersionSet`, `Version::Get`, `PickCompaction`, `CompactRange`, `LogAndApply`, used (unchanged) by our `ForceFullCompaction`.
- `db/write_batch.cc`, `db/log_writer.cc`, `db/memtable.cc` – write path internals referenced in §2.2.
- `db/builder.cc` – `BuildTable`; SSTable construction.
- `db/dbformat.h` – internal key, sequence number, value type encoding (`kTypeValue` / `kTypeDeletion`).
- `db/db_test.cc` – added `DBTest.COP290_*` tests and extended `ModelDB` for `Scan`, `DeleteRange`, `ForceFullCompaction`.